

L1

Introduction to ANN

- Knowledge prereq. $\rightarrow$ ML, Prog., Maths
- essential libraries - PyTorch, TensorFlow
 - $\hookrightarrow$ use google colab.
- ML is a field of study that gives comp. the ability to learn w/o being explicitly programmed.
 - $\hookrightarrow$ learn directly from data (no programming rules)
- some common applica^{ns} $\rightarrow$ housing price predictⁿ, autonomous vehicles, sentiment analysis, spam filtering, recommender engine,
- Focus $\rightarrow$ structured data (in tables, predict a target from features)
- Method $\rightarrow$ manual feature engg.
 - $\hookrightarrow$ human expert design features, success depends on feature quality
- Linear models $\rightarrow$ linear relaⁿ, linear, fast, but simple
- Tree models $\rightarrow$ learns if else rules, RF, non-linear powerful
- Bottleneck $\rightarrow$ feature engg., req. domain expertise, is suboptimal.
 - $\rightarrow$ enable for high-dim. data.
 - $\rightarrow$ hid. representaⁿ learning
 - $\rightarrow$ performan plateau.
- DL learns features automatically

- DL handles image, text & audio, no manual feature engineering.
- DL 'Applicaⁿ' → CV, ASR & TTS, NLP, game playing & robotics
- ML DL

manual FE

learns features

automatically

learns shallow

builds hierarchical

patterns

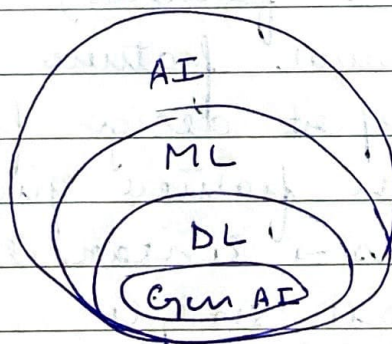
features

works on small data

better with large datasets

helps in E2E learning

- DL → big data, computational power, better algos.



L2

Intro to PyTorch & Tensors

- Tensor → DL language (somewhat)
- PyTorch → numpy on GPU, DL Research platform
- Tensor → data struct.

Scalar (0-dim) - single values (x, y, a)
vector (1-dim) $\rightarrow$ array of nos. (x, v)
matrices (2-dim) $\rightarrow$ grayscale images (X, W)
tensors $\rightarrow X \in \mathbb{R}^{d \times d}$

L3 PyTorch Installation & Env. Setup

- `import torch` (version)
- `torch.cuda.is_available()`
- - `current_device()`
 - `get_device_capability()`
 - `get_device_name()`
 - `get_device_properties()`

L4 Tensor - Creation Methods

- `scalar = torch.tensor(7)` $\rightarrow$ 0d
- `vector = torch.tensor([1, 2, 3])` $\rightarrow$ 1d
- `matrix = torch.tensor([
 [1, 2, 3],
 [4, 5, 6],
])` $\rightarrow$ 2d
- `dtype = float, torch 32`
 `var.dtype`
 `var.element_size()`

- _/_/_
- `np_array = np.array([[1, 2], [4, 5]])`
 ↳ create tensors from numpy arrays seamlessly interchange btw. them
 - create empty, ones & zeros tensors
 torch.ones(2, 4)
 ↳ 2, 4 matrix filled with ones
 or torch.full((3, 2), 42)
 - create tensors with sequential values or evenly spaced values.
 ↳ use `arange`, `linspace`
 - create identity matrices (torch.eye(3))
 3x3 identity matrix.
 - create diagonal matrices
 $v = \text{torch.tensor}([1, 2, 3])$
 $d = \text{torch.diag}(\text{vector})$
 or you can extract the diagonal from a matrix.

L5

Tensor-Manipulation Methods

- uniform random distribution
`torch.rand(size=(3, 4))`
`torch.min(var).item(): .6f`
- create an random tensor with image-like dimensions (3d)
 ↳ height, width, channels
 in practice → use `use (C, H, W)`

- create a tensor with a range of func^{ns}.
- or use already predefined shape to get the no. of os or ls.
- define tensors of different data types
 ↳ requires_grad = False
 ↳ if tensor operation on tensors gets recorded.
- op^{ns} → +, x, - (op^{ns} happens on all nos. in the tensor)

or corresponding places

$$A \times B = C$$

$m \times n \quad n \times p \quad m \times p$

$$C[i, j] = \sum_{k=1}^n A[i, k] \times B[k, j]$$

we use matrix multiplication - in neural network forward pass, convolution process, gradient computation & backpropagation
 torch.matmul(A, B) or A @ B

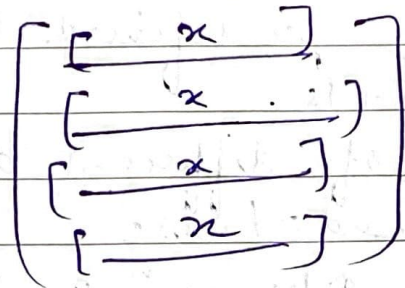
- ↳ highly optimized for PyTorch libraries
1. similarity measure → higher dot product indicates more similar vectors
 2. NN computation → linear layer output
 = input · weights

Tensor - Reshaping & Indexing Methods

x.reshape(1, 7)

//_

torch.stack([x, x, x, x], dim = ?)



dim = 0

↳ rows

dim = 1

↳ columns

- now, to reduce dimensions
 $x' = x \cdot \text{squeeze}()$

$x' \cdot \text{unsqueeze}(\text{dim} = 0) \rightarrow$ adds up along the row.

- permute (2, 0, 1)

↓
3rd axis

comes at 1st

- torch.arange(1, 10).reshape(1, 3, 3)

3d

↓
channels

matrix

$x[0] \rightarrow$ you get an matrix 3x3

$x[:, 0] \rightarrow$ first row of matrix

and entire 0th the posⁿ / dim

$x[:, 1, 1] \rightarrow$ 1th posⁿ of that row.
all of 0th 1th matrix's row

L7

PyTorch & Numpy Integration

torch.from_numpy(array)

//_

converts numpy to Tensor
convert tensor to numpy
↳ `tensor.numpy()`

L8 Reproducibility in PyTorch

- why is this imp? others could run your code, it should make results I'm claiming, others should be able to validate that.
- we use,
`RANDOM_SEED = 42`
↳ `torch.manual_seed(RANDOM_SEED)`
↳ this for CPU
For GPU,
`torch.cuda.manual_seed_all(RANDOM_SEED)`

L9 Hardware Acceleration in PyTorch

- `nvidia-smi` → to check whether cuda is available or not.
`torch.cuda.is_available()`
- no. of GPUs → `cuda.device_count()`
- current device → `torch.cuda.current_device()`
- GPU name → `torch.cuda.get_device_name(0)`

- memory allocated $\rightarrow$ cuda. memory allocated (c)
- check the version of cuda, because some open-source things run on specific version of cuda only.
- `tensor.to(device)` $\Rightarrow$ this moves your tensor to GPU.
`device = "cuda" if available else "cpu"`
- Remember, if your tensor is on GPU, then you can't convert it to numpy array. You'll have to copy the tensor to host memory first.

L10 Common PyTorch Errors

- RuntimeError $\rightarrow$ if the dim. of matmul is not fine.
- `torch.mm(t-a, t-ab)`
 $\rightarrow$ shortcut for matmul
- then error will be if you try to perform operations on 2 different types of datatype.
`torch.Tensor.type(dtype = ?)`
 $\rightarrow$ it helps to change the type of tensor.

- _/_/_
- then, there might be problem if 2 tensors are on 2 different devices.
 ↳ you have to make sure devices should be same.
 - best practice → write device agnostic code, monitor memory carefully, set defined random seeds, put robust catch except^{ns} to handle errors.

L11 LR in PyTorch: Data Creaⁿ & Loading

- get torch & matplotlib
 - define → weights & bias
- $$y = mx + c$$
- $\downarrow$ $\downarrow$
 weight bias

define first 10 x's
 & 10 y's

⇓

split your data into train & test
 now, our dataset is ready in our tensors,
 but we have to now load them.

- torch.utils.data ⇒ TensorDataset, DataLoader

```
train = TensorDataset(X-tr, y-tr)
test  = "      (X-test, y-test)
```

- we use that dataloader to get the dataset. Always put shuffle as true,

//_

so that there is no bias.

$$\text{len}(\text{train_data_loader}) = \frac{\text{len}(X_{\text{train}})}{\text{batch-size}}$$

↳ tip $\Rightarrow$ take ceil of decimal values.

- we used this, so that train & test dataset are at 1 place.
- NUM_WORKERS $\Rightarrow$ no. of subprocesses for data loading

L12

LR: Data Viz

- plot the predic^{ns} $\rightarrow$ use matplotlib
 - ↳ currently we are use scatterplot
 - ↳ train-data, train-labels, test-data, test-labels, predictions
 - ↳ once you have your y-pred, you can put that as well.
- define $\rightarrow$ legend, axis-titles, x/y labels, grid, etc for better visualization.

L13

LR: Model Definition

- import torch & torch.nn

↳ provides tools to build neural networks

- class LinearRegressionModel(torch.nn.Module):

↳ it inherits parameter mgmt. from the neural networks module

↳ now define the constructions of the class.

```
def __init__(self):
```

```
    super().__init__()
```

```
    self.wts = nn.Parameter( - - - )
```

```
    self.bias = nn.Parameter( - - - )
```

always returns

↳ requires_grad = True

the predicⁿ

```
def forward(self, x) -> Tensor:
```

```
    return self.wts * x + self.bias
```

- now call the model. Then, you can get your y-pred. Currently, they are random values. That's why we have to do the training step to learn the parameters.

L14

LR: Custom Loss funcⁿ

- class MeanAbsoluteError(torch.nn.Module)

```
    mean(abs(y_pred - y_true))
```

- define constructors & forward pass

(self, predictions, target)

torch.Tensor

shape: [batch-size, output-dim]

- class HuberLoss (nn.Module)

↳ it has quadratic loss for small errors (smooth gradient), linear for large errors (robust to outliers)

$$L_{\Delta}(a) = 0.5 a^2 \quad \text{if } \text{abs.}(a) \leq \Delta$$

$$= \Delta (\text{abs.}(a) - 0.5 \times \Delta) \quad \text{if otherwise}$$

- define constructors & forward pass to calculate the Huber loss.

↳ it should return a torch.Tensor scalar value.

- we vectorize it for each batch and not element by element.

- in this method, we learnt how to define our own custom loss func^{ion} (eg. Huber loss, MAE loss)

- you can use built-in loss func^{ion} like nn.L1Loss()(—, —)

L15 Training Loop

- use 42 manual seed, define the custom linear Reg. Model()
model.state_dict()
 ↳ lists the named params.
- to make predic^{ns} with model
with torch.inference_mode():
 y_preds = model(X_test)
- currently, we have the training loop ready and customized loss function. But if you remember, the predictions we are currently getting are based on the random initializa^{ns} of our parameters. So now we require a optimizer (like gradient descent) is needed to train our params.
- opt = torch.optim.SGD(
 params = model.parameters(),
 lr = 0.001
) ⇒ configure the optimiza^{ns} algo.
- no. of epochs → no. of times the model will pass over the training data.
model.train() ⇒ puts in training mode.
y_pred = model(X_train)
loss = loss_fun(y_pred, y_train)
optimizer.zero_grad()
loss.backward()
optimizer.step() ⇒ upda^{ns}

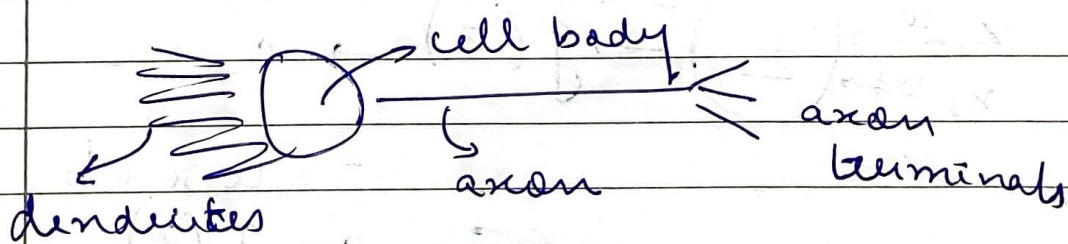
- _/_/_
- similar now define testing loop (`eval-model()`)
`model.eval()`
 with `torch.inference_mode()`
 $\Rightarrow$ do the predictions
 use test loss funcⁿ to calculate the loss.
 - keep track of variables for the loss curve $\Rightarrow$ you can print them & also visualize them.
 - when you put the model in eval mode, it basically disables the gradient computa^{ns}
 - the avg. training is calculated for the epoch.
 - after training, you can use `model.state_dict()` $\rightarrow$ to see the final values of your learned parameters.

Lib Model Serializaⁿ in PyTorch

- from `pathlib` import `Path`
`path = Path("models")`
 $\hookrightarrow$ `parents=True`, `exist_ok=True`
- naming convenⁿ for model $\Rightarrow$ '.pth'
- Model save path = model path / model name

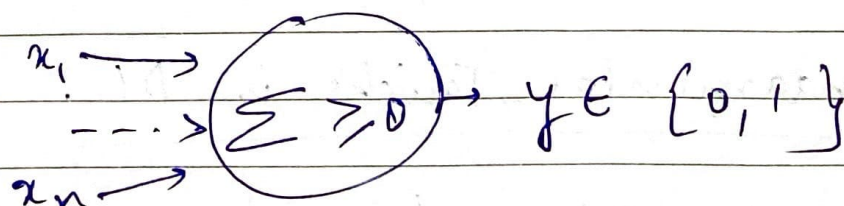
- _/_/_
- how to save the model weights
`torch.save(obj = model.state_dict(),
| = MODEL-SAVE-PATH)`
 - you can perform the sanity check by
 checking file size (`.st-size()`)
 - you can also get (load) the model back
 and examine it.
`torch.load(MODEL-SAVE-PATH, weights_
only = True)`
 → then we can print the parameter
 names & its corresponding values.

L17 From Biology to Models



receive signals via dendrites, sends output
 signal via axon.

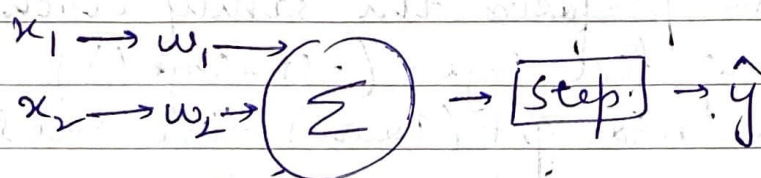
- MP Neuron



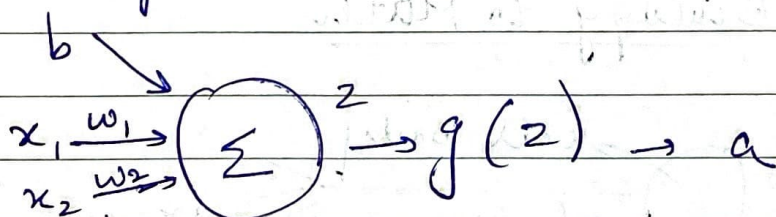
θ is fixed

networks
 are simple units

- MP Neuron $\rightarrow$ no weights, no learning
 Δ it is restricted to binary I/O.
- Perceptron



- $\rightarrow$ we have lots noise
- $\rightarrow$ we have learning rule to introduce backpropagation
- $\rightarrow$ convergence guarantee (if linearly separable)
- $\rightarrow$ not applicable for non-linear ones.
- $\rightarrow$ very harsh threshold.



$$z = w \cdot x + b$$

$\leftarrow a = g(z)$
 non-linear activation

The smooth activation function gives us a gradient.

L18

Fundamental Tricks in DL.

- normalizing $\rightarrow$ computers only calculate, they can't perceive

- everything has to be converted into a proper number.
- eg. of self-driving car
 - input → what car sees
(camera, motion data)
 - output → what the car does
(control actions, other predictions)
- now the problem is how we convert these abstract categories into vectors
 1. one-hot encoding (inefficient when too many classes)
 2. embedding layers (learns a dense, meaningful vector for each class)

- funcⁿ tricks

- our problem is data (I/O)
- solⁿ is funcⁿ of $I \rightarrow O$.
- task is to find that solⁿ.
- every funcⁿ has 2 parts
 - architecture
 - parameters (w) $\Rightarrow$ learns from data.

- parameterizaⁿ tricks

- the goal of our training is to find the optimal value of θ .

$$\hat{y} = f(x; \theta)$$

- now our problem becomes optimizaⁿ satisfacⁿ funcⁿ.

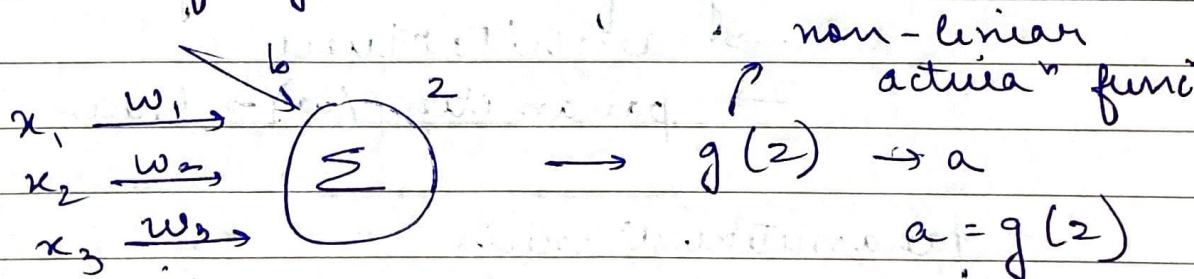
1. measure error using a loss funcⁿ
 $L(y, \hat{y}) = L(y, f(x; \theta)); L \uparrow$, sad
2. find θ^* that gives lowest possible loss.

L19 Feed Forward Network Architecture

- Input layer $\rightarrow$ receives the data and pass the value.
- Hidden layer $\rightarrow$ network's computational engine, they learn the abstract features.
- Output layer $\rightarrow$ gives the predicⁿ

$I \rightarrow H \rightarrow O$

Feed forward forward (one-direction)
fully connected



$$\Sigma = x_1 w_1 + x_2 w_2 + \dots + b$$

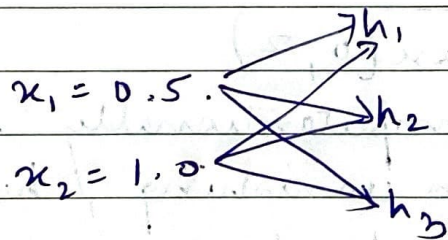
- linear sum + non-linear "activation"

- so, calculating things neuron 1-by-1 is very inefficient. So we compute an entire layer at once using vectorized ops (like matmul)

$$a^{[l-1]} \xrightarrow{W^{[l]}, b^{[l]}} a^{[l]}$$

$$\left. \begin{aligned} z^{[l]} &= W^{[l]} a^{[l-1]} + b^{[l]} \\ a^{[l]} &= g(z^{[l]}) \end{aligned} \right\} \begin{array}{l} \text{this is} \\ \text{computational} \\ \text{efficient} \\ \& \text{ perfect for GPUs} \end{array}$$

L20 FFN Arch. Ex.



$$g(z) = \max(0, z) \rightarrow \text{ReLU}$$

$$x = \begin{bmatrix} 0.5 \\ 1 \end{bmatrix}_{2 \times 1}$$

$$W = \begin{bmatrix} 0.2 & 0.7 \\ -0.4 & 0.1 \\ 0.9 & -0.3 \end{bmatrix}_{3 \times 2}$$

$$z = Wx + b$$

$$= \begin{bmatrix} 0.9 \\ 0.1 \\ -0.35 \end{bmatrix}$$

$$b = \begin{bmatrix} 0.1 \\ 0.2 \\ -0.5 \end{bmatrix}$$

$$a = g(z) \rightarrow \text{apply ReLU} \Rightarrow \begin{bmatrix} 0.9 \\ 0.1 \\ 0 \end{bmatrix}$$

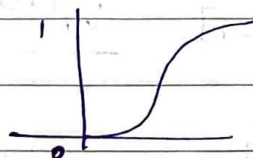
L2)

Activation Functions

- only linear layers $\rightarrow$ model collapses into a single linear layer itself w/o a non-linear activation function.

- sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

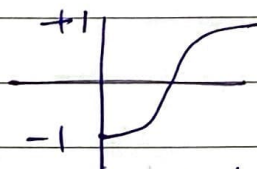


neuron's
"firing rate"

problem of vanishing gradients & not 0-centered.

- tanh

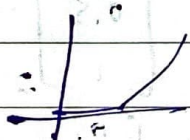
$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$



0 centered but vanishing gradient

- ReLU

$$\text{ReLU}(z) = \max(0, z)$$



computationally efficient,
~~non~~ vanishing, but dying
ReLU problem.

- Leaky ReLU / Parametric ReLU

$$f(z) = \begin{cases} z & z > 0 \\ \alpha z & z \leq 0 \end{cases}$$

allows dead
neurons to survive

- Gelu $\rightarrow$ smoother probabilistic alternative
- Output $\rightarrow$ sigmoid, or softmax (multiclass),
none (for regression)
- now, tanh or sigmoid aren't used in very deep hidden layers. Only used in output layers.